qp primitive catalog

Generated from the registry by `tools/gen_catalog.py` — do not edit by hand.

71 primitives · 71 approved (as of 2026-10-02 01:21 UTC). Approval status changes as verification progresses; the source of truth is `approvals/approvals.json`, queried at runtime via `qp.approved_primitives()`.

Conventions:

- **input** `source` — takes a 1-D float array (pick close/open/high/low/hl2/hlc3/ohlc4/volume/body_high/body_low); **input** `bars` — takes a `qp.Bars` (validated OHLCV DataFrame).
- **output** `value` — returns one `np.ndarray` aligned to the input bars; anything else — returns `dict[str, np.ndarray]` with those keys.
- NaN marks warm-up bars and bars where the value is undefined (outside session, before an anchor, level not yet known).

candle

✅ `candle.bar_range`

Full bar range, high - low.

- input: `bars`
- output: single series
- params: none

✅ `candle.body`

Absolute candle body, |close - open|.

- input: `bars`
- output: single series
- params: none

✅ `candle.lower_wick`

Lower wick, min(close, open) - low.

- input: `bars`
- output: single series
- params: none

✅ `candle.upper_wick`

Upper wick, high - max(close, open).

- input: `bars`
- output: single series
- params: none

extremes

✅ `extremes.highest`

Highest value of `source` over the last `length` bars (inclusive of current bar). Matches Pine `ta.highest`.

- input: `source`
- output: single series
- params: `length`: int = 20 (min 1)

✅ `extremes.lowest`

Lowest value of `source` over the last `length` bars (inclusive of current bar). Matches Pine `ta.lowest`.

- input: `source`
- output: single series
- params: `length`: int = 20 (min 1)

levels

✅ `levels.day_open`

Today’s RTH open (09:30 ET bar). NaN before today’s open exists.

- input: `bars`
- output: single series
- params: none

✅ `levels.dynamic_sr`

Dynamic S/R levels — port of “Support Resistance Dynamic v2”. Clusters the last `max_pivots` confirmed pivots into channels no wider than (300-bar range * channel_width_pct /100), ranks by pivot count, keeps the top `max_levels`. Returns {sr1…srN}, strongest first. `source`: “hl” uses high/low pivots, “body” uses max/min(open,close). Needs ~300 bars of history before the first level.

- input: `bars`
- output: dict with keys `sr1, sr2, sr3, sr4, sr5, sr6`
- params: `pivot_period`: int = 10 (min 4, max 30) · `max_pivots`: int = 20 (min 5, max 100) · `channel_width_pct`: float = 10.0 (min 1) · `max_levels`: int = 6 (min 1, max 10) · `min_strength`: int = 3 (min 1, max 10) · `source`: str = 'hl'

✅ `levels.monday_high`

This ISO week's Monday RTH high: the running high on Monday, then constant Tuesday-Friday. NaN until Monday RTH has data.

- input: `bars`
- output: single series
- params: none

✅ `levels.monday_low`

This ISO week's Monday RTH low.

- input: `bars`
- output: single series
- params: none

✅ `levels.monthly_high`

This ET month's running RTH high.

- input: `bars`
- output: single series
- params: none

✅ `levels.monthly_low`

This ET month's running RTH low.

- input: `bars`
- output: single series
- params: none

✅ `levels.monthly_open`

This ET month's RTH open.

- input: `bars`
- output: single series
- params: none

✅ `levels.pm_high`

Premarket high (04:00-09:30 ET). Running during PM; frozen through the rest of the day. Needs extended-hours bars in the feed (Alpaca IEX includes them).

- input: `bars`
- output: single series
- params: none

✅ `levels.pm_low`

Premarket low (04:00-09:30 ET), frozen after 09:30.

- input: `bars`
- output: single series
- params: none

✅ `levels.postmarket_high`

Post-market high — window 16:00 → 04:00 next day ET (NY close → premarket start). This is the “overnight” session in the 24h model NY/post/pre; premarket is EXCLUDED. Running through the window, then frozen across the next RTH day. Needs extended-hours bars (polygon / hybrid), All-day view.

- input: `bars`
- output: single series
- params: none

✅ `levels.postmarket_low`

Post-market low — window 16:00 → 04:00 next day ET (NY close → premarket start, premarket excluded).

- input: `bars`
- output: single series
- params: none

✅ `levels.postpre_high`

Post+Pre high — window 16:00 → 09:30 next day ET: the full extended range from the NY close to the next open (post-market 16:00-04:00 + premarket 04:00-09:30). Running through the window, frozen across the RTH day. This is the old “overnight_high” renamed. The user’s Pine used 18:00-09:30 (CME Globex); here it anchors to the 16:00 NY equity close. Needs extended-hours bars (polygon / hybrid), All-day view.

- input: `bars`
- output: single series

- params: none

✅ `levels.postpre_low`

Post+Pre low — window 16:00 → 09:30 next day ET (post-market + premarket, the full close→open range).

- input: `bars`
- output: single series
- params: none

✅ `levels.prev_day_close`

Yesterday’s RTH close, held constant across today.

- input: `bars`
- output: single series
- params: none

✅ `levels.prev_day_high`

Yesterday’s RTH high, held constant across today.

- input: `bars`
- output: single series
- params: none

✅ `levels.prev_day_low`

Yesterday’s RTH low.

- input: `bars`
- output: single series
- params: none

✅ `levels.prev_day_open`

Yesterday’s RTH open.

- input: `bars`
- output: single series
- params: none

✅ `levels.prev_month_open`

Previous month’s RTH open (PMO). Fetch window must reach into last month.

- input: `bars`

- output: single series
- params: none

✅ `levels.prev_week_open`

Previous ISO week's RTH open (PWO). Fetch window must reach into last week.

- input: `bars`
- output: single series
- params: none

✅ `levels.prev_year_open`

Previous year's RTH open (PYO). Fetch window must reach into last year.

- input: `bars`
- output: single series
- params: none

✅ `levels.today_high`

Today's running RTH high so far.

- input: `bars`
- output: single series
- params: none

✅ `levels.today_low`

Today's running RTH low so far.

- input: `bars`
- output: single series
- params: none

✅ `levels.today_vol_max`

Today's running RTH maximum single-bar volume so far (the volume analogue of today_high). Powers "one of the day's biggest volume bars" checks.

- input: `bars`
- output: single series
- params: none

✅ `levels.weekly_high`

This ISO week's running RTH high.

- input: `bars`
- output: single series
- params: none

✅ `levels.weekly_low`

This ISO week's running RTH low.

- input: `bars`
- output: single series
- params: none

✅ `levels.weekly_open`

This ISO week's RTH open (Monday 09:30 ET bar).

- input: `bars`
- output: single series
- params: none

✅ `levels.window_high`

High of an intraday ET time window [start, end) in hhmm (default 930–945 = the OPENING RANGE), frozen for the rest of the day. Set 1500–1600 for power-hour high, etc. Break of it = ORB long trigger.

- input: `bars`
- output: single series
- params: `start`: int = 930 (min 0, max 2359) · `end`: int = 945 (min 0, max 2359)

✅ `levels.window_low`

Low of an intraday ET time window [start, end) in hhmm (default 930–945 = the OPENING RANGE), frozen for the rest of the day. Break of it = ORB short trigger.

- input: `bars`
- output: single series
- params: `start`: int = 930 (min 0, max 2359) · `end`: int = 945 (min 0, max 2359)

✅ `levels.yearly_open`

This year's RTH open (first RTH bar of the year — the S/R script's YO line). The fetch window must include January for a real value.

- input: `bars`
- output: single series
- params: none

ma

✅ `ma.ema`

Exponential MA. Matches TradingView `ta.ema(source, length)` — SMA-seeded, α=2/(length+1). NaN for first (length-1) bars, seed = SMA(length) at bar (length-1), then α-recurrence forward.

- input: `source`
- output: single series
- params: `length`: int = 9 (min 1)

✅ `ma.hma`

Hull MA. Matches TradingView `ta.hma(source, length)` — wma(2*wma(src, round(len/2)) - wma(src, len), round(sqrt(len))). Rounds half away from zero like Pine math.round (matters for odd lengths, e.g. 15 → half=8).

- input: `source`
- output: single series
- params: `length`: int = 20 (min 2)

✅ `ma.pine_5day`

5-day MA: SMA of close over the last `length` 1-minute RTH bars (default 1950 = 6.5h x 5 RTH days). Matches Pine `request.security(sym,"1",ta.sma(close,1950))`. Always computed on 1-minute data (compute_tf='1m') regardless of the chart timeframe, and held flat across non-RTH bars — so the line is identical whether you view a 1m, 5m or 15m chart.

- input: `bars`
- output: single series
- params: `length`: int = 1950 (min 1)

✅ `ma.rma`

Wilder's smoothed MA. Matches TradingView `ta.rma(source, length)` — α=1/length, seeded with SMA(length). Basis for `ta.rsi` and `ta.atr`.

- input: `source`
- output: single series
- params: `length`: int = 14 (min 1)

✅ `ma.sma`

Simple moving average of `source` over `length` bars. Matches TradingView `ta.sma(source, length)` — NaN for the first (length-1) bars, mean of the trailing window afterwards.

- input: `source`

- output: single series
- params: `length`: int = 9 (min 1)

✅ `ma.vwma`

Volume-weighted MA, `sum(source*volume)/sum(volume)` over `length` bars. Matches TradingView `ta.vwma(source, length)`. Source picked via `source` param (default close).

- input: `bars`
- output: single series
- params: `length`: int = 20 (min 1) · `source`: str = 'close'

✅ `ma.wma`

Weighted MA — linear weights 1…length. Matches TradingView `ta.wma(source, length)`.

- input: `source`
- output: single series
- params: `length`: int = 9 (min 1)

osc

✅ `osc.rsi`

Wilder RSI. Matches TradingView `ta.rsi(source, length)` — rma-smoothed gains/losses, `100 - 100/(1 + avg_gain/avg_loss)`. Value in [0,100]. NaN for first `length` bars.

- input: `source`
- output: single series
- params: `length`: int = 14 (min 1)

pivots

✅ `pivots.floor`

Floor pivots {P, R1, R2, R3, S1, S2, S3} from yesterday's H/L/C. session='eth' (default) uses the full 24h day — matches the user's Pine "Full Day Daily Pivots (ETH)" which requests the 24h Globex daily bar. session='rth' uses the 09:30-16:00 daily bar (a standard TV equities daily).

- input: `bars`
- output: dict with keys `P, R1, R2, R3, S1, S2, S3`
- params: `session`: str = 'eth'

structure

✅ `structure.pivot_high`

Pivot high — matches Pine `ta.pivothigh(source, left, right)`. Returns the pivot's price on the confirmation bar (right bars after the pivot), NaN elsewhere.

- input: `source`
- output: single series
- params: `left`: int = 10 (min 1) · `right`: int = 10 (min 1)

✅ `structure.pivot_low`

Pivot low — matches Pine `ta.pivotlow(source, left, right)`.

- input: `source`
- output: single series
- params: `left`: int = 10 (min 1) · `right`: int = 10 (min 1)

trend

✅ `trend.slope`

Trend-slope STRENGTH of the source over `length` bars: net modelled move ÷ the noise around the fitted line. ~0 in chop, 2-4 in a shallow drift, 10+ in a clean steep trend (capped ±99). This is the exact number the strategy rising/falling test thresholds — plot it to pick your threshold by eye.

- input: `source`
- output: single series
- params: `length`: int = 12 (min 4)

volatility

✅ `volatility.atr`

Average True Range (Wilder). Matches TradingView `ta.atr(length)` — rma(true_range, length).

- input: `bars`
- output: single series
- params: `length`: int = 14 (min 1)

✅ `volatility.atr_daily`

Daily ATR — the Wilder ATR computed on 1-DAY bars, always, regardless of the chart timeframe (compute_tf='1d'), then held flat across each day. Matches Pine `request.security(syminfo.tickerid, "D", ta.atr(length))`. Lets you read the DAILY ATR while looking at an intraday chart. Needs enough daily history for the warm-up — raise Days to >= ~3x length (e.g. Days 30-45 for length 14).

- input: `bars`
- output: single series
- params: `length`: int = 14 (min 1)

✅ `volatility.bb`

Bollinger Bands. Returns {middle, upper, lower} — middle = sma(source, length), upper/lower = middle ± mult * stdev(source, length). Matches TradingView `ta.bb(source, length, mult)`.

- input: `source`
- output: dict with keys `middle, upper, lower`
- params: `length`: int = 20 (min 2) · `mult`: float = 2.0 (min 0)

✅ `volatility.bb_ema`

Bollinger Bands with an EMA middle line — matches the L3 script's `l3_bb_ema = ta.ema(close, length)` + `ta.stdev(close, length)` bands. Returns {middle, upper, lower}.

- input: `source`
- output: dict with keys `middle, upper, lower`
- params: `length`: int = 21 (min 2) · `mult`: float = 2.0 (min 0)

✅ `volatility.stdev`

Population standard deviation (ddof=0) of `source` over `length` bars. Matches TradingView `ta.stdev(source, length)`.

- input: `source`
- output: single series
- params: `length`: int = 20 (min 2)

✅ `volatility.true_range`

True range, `max(H-L, |H-prev_close|, |L-prev_close|)`. Matches TradingView `ta.tr(handle_na=true)`: the first bar (no prior close) falls back to high-low. This is the variant `ta.atr` builds on.

- input: `bars`
- output: single series

- params: none

volume

✅ `volume.avg_volume`

Average DAILY volume over the last `length` days. Computed on 1-day bars (compute_tf='1d') so it is the same value on any chart timeframe — matches `ta.sma(volume, length)` on a daily chart. Held flat across intraday bars. Raise Days so the warm-up (>= length daily bars) is available.

- input: `bars`
- output: single series
- params: `length`: int = 20 (min 1)

✅ `volume.rel_volume`

Intraday Relative Volume (RVOL). For each bar: cumulative volume from the RTH open (09:30 ET) through the bar, divided by the AVERAGE cumulative volume at the same time-of-day over the previous `length` RTH sessions. ~1.0 = normal for this hour, 2.0 = twice the usual volume by now. Matches the day-trader / TradingView intraday Relative Volume. Non-RTH bars are NaN. Needs `length` prior RTH days of history — raise Days (e.g. Days >= length + a few). TIMEFRAME-INVARIANT: because it is cumulative volume from the session open (the same total whether you sum 1m or 5m bars), the value at any given time is identical on a 1m/5m/15m/1h chart — verified to 1e-15. Changing the chart TF does not change it.

- input: `bars`
- output: single series
- params: `length`: int = 20 (min 1)

vwap

✅ `vwap.anchored`

AVWAP from a user-chosen datetime — the cluster script's Earnings/News VWAP A/B. Anchors at the first bar whose timestamp >= `anchor` (Pine: `time >= i_earnDate and time[1] < i_earnDate`). `anchor` is an ET datetime string, e.g. "2026-07-01 09:30". Empty anchor → all NaN.

- input: `bars`
- output: single series
- params: `anchor`: str = '' · `rth_only`: bool = True

✅ `vwap.gap`

AVWAP anchored at the most recent gap bar: `|open - prev_close| >= atr(atr_length) * atr_mult`. With rth_only (default) prev_close is the previous RTH bar, so the 09:30 bar carries the full overnight gap — matching Pine on an RTH chart.

- input: `bars`
- output: single series
- params: `atr_length`: int = 14 (min 1) · `atr_mult`: float = 1.5 (min 0) · `rth_only`: bool = True

✅ `vwap.last_hour_hh`

AVWAP seeded each new ET day with yesterday's last-hour (15:00-16:00 ET by default) highest-high bar, then accumulating today's bars.

- input: `bars`
- output: single series
- params: `last_hour_start`: int = 15 (min 10, max 15) · `rth_only`: bool = True

✅ `vwap.last_hour_ll`

AVWAP seeded from yesterday's last-hour lowest-low bar.

- input: `bars`
- output: single series
- params: `last_hour_start`: int = 15 (min 10, max 15) · `rth_only`: bool = True

✅ `vwap.monthly`

Monthly VWAP — resets on the first (RTH) bar of each ET calendar month. Matches Pine `ta.vwap(hlc3, isNewMonth)` on an RTH chart.

- input: `bars`
- output: single series
- params: `rth_only`: bool = True

✅ `vwap.nday_block`

N-session BLOCK VWAP — exact Pine `ta.vwap(hlc3, isNewDay and dCount % N == 0)`. Accumulates across N sessions then RESETS: on a block-start session it equals the plain session VWAP, and by the Nth session it spans all N. Use this to reproduce the TradingView study that uses that Pine. For a VWAP that is ALWAYS anchored at yesterday's open (never resets to one day) use `nday_rolling` instead. PHASE: Pine's dCount counts sessions from wherever the chart history starts, so which sessions are block-starts is arbitrary. `anchor_offset` (0…N-1) shifts the phase — if the reset days don't line up with your TradingView, try 0 then 1 (for N=2) until they match, then keep Days fixed.

- input: `bars`

- output: single series
- params: `n_days`: int = 2 (min 1) · `rth_only`: bool = True · `anchor_offset`: int = 0 (min 0)

✅ `vwap.nday_rolling`

N-session ROLLING (trailing) VWAP — always anchored at the OPEN of the session (N-1) ago through the current bar. For N=2 that is “anchored from yesterday’s open until now”: every bar’s value is the volume-weighted average price over the last N sessions (yesterday + today). The anchor slides forward each day; it NEVER resets to a single day. This is NOT the Pine `dCount%N` study — for that use `nday_block`. Only the first (N-1) sessions of the fetched window degenerate to a shorter span (nothing earlier exists). To eyeball on TradingView, drop an Anchored VWAP at yesterday’s 09:30.

- input: `bars`
- output: single series
- params: `n_days`: int = 2 (min 1) · `rth_only`: bool = True

✅ `vwap.session`

Session VWAP for US equities. Resets at 09:30 ET, computes only during RTH (09:30-16:00 ET), source = HLC/3. Matches the TradingView built-in VWAP on an RTH chart. On daily+ frames each bar is its own session (value = HLC/3). Verify on 1m/5m/15m — Alpaca hourly bars are clock-aligned (09:00) unlike TV’s session-aligned (09:30) hourly bars.

- input: `bars`
- output: single series
- params: none

✅ `vwap.stdev_bands`

Session VWAP with ± mult * (running stdev of price around the VWAP). Returns {middle, upper, lower}. This is the BBZ script’s `bz_vwap_stdev` (“SDV”) — NOT the same as `volatility.stdev`, which is a rolling-window stdev of close.

- input: `bars`
- output: dict with keys `middle, upper, lower`
- params: `mult`: float = 1.0 (min 0)

✅ `vwap.swing_hh`

AVWAP anchored at the most recently *confirmed* swing high — `ta.pivothigh(high, lookback, lookback)`. Confirmation lags by `lookback` bars, matching Pine.

- input: `bars`
- output: single series
- params: `lookback`: int = 25 (min 2) · `rth_only`: bool = True

✅ `vwap.swing_ll`

AVWAP anchored at the most recently confirmed swing low.

- input: `bars`
- output: single series
- params: `lookback`: int = 25 (min 2) · `rth_only`: bool = True

✅ `vwap.today_hh`

AVWAP from today's intraday highest-high bar. Re-anchors whenever a new HH prints during the ET day. Matches the VWAP-Cluster script's `vwap_hh`.

- input: `bars`
- output: single series
- params: `rth_only`: bool = True

✅ `vwap.today_ll`

AVWAP from today's intraday lowest-low. Re-anchors on new LL.

- input: `bars`
- output: single series
- params: `rth_only`: bool = True

✅ `vwap.week_hh`

AVWAP from this ISO-week's highest bar. Re-anchors on new HH.

- input: `bars`
- output: single series
- params: `rth_only`: bool = True

✅ `vwap.week_ll`

AVWAP from this ISO-week's lowest bar. Re-anchors on new LL.

- input: `bars`
- output: single series
- params: `rth_only`: bool = True

✅ `vwap.weekly`

Weekly VWAP — resets on the first (RTH) bar of each ISO week. Matches Pine `ta.vwap(hlc3, isNewWeek)` on an RTH chart.

- input: `bars`
- output: single series

- params: `rth_only`: bool = True